# Our Method, Mechanism by Mechanism

A study guide: what each part does, how it does it, the mathematics, and why it is there

22 October 2026

## Contents


**1 The idea in one page** **2**

**2 The skeleton we build on** **2**

**3 Foundation: why exact sums are the right tool** **3**

**4 Mechanism 1: the class-statistics release** **4**

**5 Mechanism 2: the separation statistic $F$ and the gate** **4**

**6 Mechanism 3: the mean-anchored generator** **5**

**7 Mechanism 4: class rows with holder-weighted averaging** **6**

**8 Mechanism 5: moment correction** **7**

**9 Mechanism 6 (optional): kernel herding** **8**

**10 Mechanism 7: logit adjustment in the real phase** **9**

**11 Mechanism 8: consensus soft labels on synthetic features** **10**

**12 Mechanism 9: the exact readout** **11**

**13 Optional: noise on the released sums** **12**

**14 Putting it together** **13**

**15 Cheat sheet** **14**

# 1 The idea in one page

**The setting.** $K$ clients hold private, long-tailed data: a few classes are common and many are rare, and every client sees a different class mix. Clients run *different* model architectures, so their parameters cannot simply be averaged. Our method lives inside a three-stage pipeline (Section 2): a small shared feature extractor, a shared generator of synthetic features, and one private model ("head") per client.

**The problem.** Under long-tailed clients, three things go wrong. (1) The generator forgets rare classes. (2) Each head's last layer is fitted to its own client's class mix. (3) Synthetic features that do not carry their class label actively harm the heads.

**The principle.** *Wherever the data decide the answer, use a statistic that secure aggregation computes exactly (a sum over clients) instead of a parameter that averaging only estimates.* Sums do not depend on how the data are split across clients, so they equal what a centralised learner would compute.

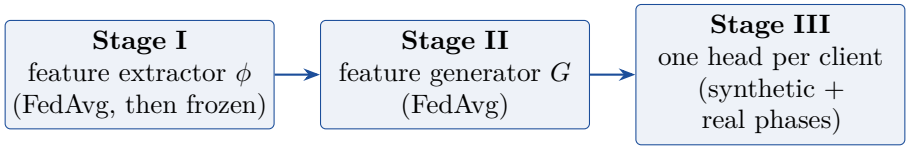
**The mechanisms**, in the order they act:

| Mechanism | Stage | Problem it solves |
|---|---|---|
| Class-statistics release (Section 4) | after Stage I | gives the server exact class means and spreads |
| Separation $F$ and the gate (Section 5) | before Stage II | decides which generator can work on this data |
| Mean-anchored generator (Section 6) | Stage II | removes the parameters that collapse for rare classes |
| Class rows + holder-weighted averaging (Section 7) | Stage II, if $F < 0.05$ | keeps class rows when means are uninformative |
| Moment correction (Section 8) | sampling | puts every class's samples at the exact mean and spread |
| Kernel herding, *optional* (Section 9) | sampling | selects samples closest to the exact class fingerprint |
| Logit adjustment (Section 10) | Stage III, real data | stops the real phase erasing what synthetic data taught |
| Consensus soft labels (Section 11) | Stage III, synthetic data | lowers the label error of imperfect synthetic features |
| Exact readout (Section 12) | after Stage III | replaces each biased last layer by the centralised one |

Section 14 assembles them into one algorithm, and Section 15 lists every symbol and setting.

# 2 The skeleton we build on

Only the parts of the pipeline that our mechanisms touch are described here.




- **Features.** Every image $x$ is mapped by the frozen extractor to a feature $h = \phi(x)$ (for MNIST a $3 \times 16 \times 16$ tensor, 768 numbers; for CIFAR $10 \times 16 \times 16$). Everything after Stage I works on features.

- **Generator.** A conditional variational autoencoder (CVAE): an encoder $q(z \mid h, y)$, a decoder that maps a latent code $z \in \mathbb{R}^{16}$ (50 for CIFAR) and a class $y$ to a synthetic feature $\tilde{h}$. It is trained by FedAvg: each round every client trains its copy for 5 local epochs (Adam, learning rate $10^{-3}$, weight decay $10^{-3}$) and the server averages the copies.
- **Heads.** Client $k$ owns a head $h_k$ of architecture $a(k)$: a *body* that maps $h$ to an embedding $e$, and a *last layer* $f(e) = We + b$ giving $C$ logits. Each Stage-III round, the client trains the head for $T_s$ epochs on synthetic features and then $T_r = 5$ epochs on its own real features (SGD, learning rate 0.1). Heads of the same architecture are averaged.

**Notation.** $n_{kc}$ is client $k$'s number of class-$c$ samples, $n_c = \sum_k n_{kc}$, $N = \sum_c n_c$, $\pi_{kc} = n_{kc}/n_k$ is client $k$'s class prior. $H_c$ is the set of $m_c$ clients that hold class $c$.

# 3 Foundation: why exact sums are the right tool

Two short results justify every mechanism that uses sums. Read them first; the rest of the guide refers back to them.

## 3.1 The client split is a pure label shift

**Result 1** (pure label shift)**.** *For every client $k$ and class $c$ with $n_{kc} > 0$, client $k$'s class-$c$ samples are a uniformly random subset of the pooled class-$c$ samples. Hence $P_k(x \mid y) = P(x \mid y)$, and, because $\phi$ is shared, $P_k(h \mid y) = P(h \mid y)$.*

*Why it holds.* The split decides *how many* samples of each class a client receives, then takes them from a random permutation of that class. A random chunk of a random permutation is a uniformly random subset.

*What it buys.* Clients differ only in *how much* of each class they hold, never in *what* a class looks like. So for any class-conditional quantity (a class mean, a class spread, a class row of the generator, a readout) every client's data point at the *same* target as the pooled data. Federation can cost weighting, variance and speed, but it cannot bias such a target. A bias can only come from (i) the optimiser, (ii) parameters tied to a client's own class prior (biases and last layers), or (iii) shared parameters trained on prior-weighted losses (equally prior-weighted centrally). The mechanisms below remove (i) and (ii).

## 3.2 Sums are partition-invariant

**Result 2** (partition invariance)**.** *Let a quantity be a function $\Psi(S)$ of a sum $S = \sum_k \sum_{i \in D_k} T(x_i, y_i)$ with $T$ fixed in advance. Its value is the same for every way of splitting the pooled data across clients, and equals the centralised value.*

*Proof.* Addition is associative and commutative. □

The result is trivial; its power is in sorting the pipeline. Anything computed from sums (class means, spreads, fingerprints, the exact readout) is *exactly* what a centralised learner would compute. Anything trained by averaging (the generator, the bodies) is not. Our method moves as many decisions as possible into the first group.

## 3.3 Secure aggregation

Under secure aggregation each client uploads a masked vector; the masks cancel in the sum, so the server learns only $\sum_k v_k$ and nothing about any single $v_k$. Every release in this guide is such a sum.

# 4 Mechanism 1: the class-statistics release

**Purpose**

Give the server the exact mean and spread of every class's features, computed over all clients, without any client revealing its own data or its own class counts.

**Procedure**

Once, after Stage I (the extractor is frozen):

1. Client $k$ computes, for every class $c$: the count $n_{kc}$, the feature sum $S_{kc} = \sum_{i \in D_k, y_i = c} h_i$ (a vector), and the sum of squared norms $Q_{kc} = \sum_{i \in D_k, y_i = c} \|h_i\|^2$ (a number).
2. Secure aggregation returns $n_c = \sum_k n_{kc}$, $S_c = \sum_k S_{kc}$, $Q_c = \sum_k Q_{kc}$.
3. The server computes the class mean and the class spread (mean squared distance to the mean):

$$m_c = \frac{S_c}{n_c}, \qquad V_c = \mathbb{E}\|h - m_c\|^2 = \frac{Q_c}{n_c} - \|m_c\|^2.$$

*Derivation of $V_c$.* $\mathbb{E}\|h - m_c\|^2 = \mathbb{E}\|h\|^2 - 2\langle \mathbb{E}h, m_c \rangle + \|m_c\|^2 = \mathbb{E}\|h\|^2 - \|m_c\|^2$, and $\mathbb{E}\|h\|^2 = Q_c/n_c$.

**Worked example 1.** A class has 3 samples in total, two on client 1 and one on client 2, with (two-dimensional) features $(1,0)$, $(3,0)$ and $(2,3)$. Client 1 sends $n = 2$, $S = (4,0)$, $Q = 1 + 9 = 10$; client 2 sends $n = 1$, $S = (2,3)$, $Q = 13$. The server receives $n = 3$, $S = (6,3)$, $Q = 23$, so $m = (2,1)$ and $V = 23/3 - 5 = 8/3$. Computing the mean and spread directly on the three points gives the same numbers, as Result 2 says it must.

**Cost.** $C(d+2)$ numbers per client, once ($d$ = feature size). By Results 1 and 2, $m_c$ and $V_c$ are exactly the centralised values.

**What it does not do**

For a class with very few samples the release is close to an average over a handful of people, so it can leak whether someone is in the data. If that matters, clip features and add Gaussian noise to the sums (Section 13).

# 5 Mechanism 2: the separation statistic $F$ and the gate

**Purpose**

Decide, from the released sums alone, whether class means are far enough apart to tell classes apart. If they are, the mean-anchored generator (Section 6) is used; if not, the generator keeps learned class rows protected by holder-weighted averaging (Section 7).

**Definition.** With $\bar{m} = \frac{1}{C}\sum_c m_c$,

$$F = \frac{\frac{1}{C}\sum_c \|m_c - \bar{m}\|^2}{\frac{1}{C}\sum_c V_c} \quad = \quad \frac{\text{average squared distance of a class mean from the centre}}{\text{average within-class spread}}.$$

In units of the within-class spread, two class means are on average about $\sqrt{2F}$ apart.

**Why the decision is needed.** The mean-anchored generator receives a class only through its mean $m_y$. If two classes have nearly the same mean, it cannot produce different samples for them:

**Result 3** (what a mean anchor can express)**.** *Let the class-$c$ sampler be $X_c = \mathrm{ReLU}(m_c + g(Z, Am_c))$, with the noise $Z$ shared by all classes, $A$ a class-shared matrix and $g$ $L$-Lipschitz in its second*

*argument. Then for any two classes*

$$W_1(G_c, G_{c'}) \le (1 + L\|A\|_2)\,\|m_c - m_{c'}\|.$$

*Proof.* Use the same $Z$ for both classes. ReLU is 1-Lipschitz, so $\|X_c - X_{c'}\| \le \|m_c - m_{c'}\| + \|g(Z, Am_c) - g(Z, Am_{c'})\| \le (1 + L\|A\|_2)\|m_c - m_{c'}\|$; take expectations. □

($W_1$ is the earth-mover distance: the average distance samples must move to turn one class's distribution into the other's.) If $\|m_c - m_{c'}\|$ is tiny relative to the spread, the synthetic classes are nearly identical, and no amount of training changes that.

**Decision rule**

Compute $F$ from the released sums. If $F \ge 0.05$: use the mean-anchored generator. If $F < 0.05$: use class rows with holder-weighted averaging.

**Measured values.** $F$ is 0.25 or more on MNIST and FashionMNIST, 0.18–0.25 on CIFAR-100, 0.054–0.080 on CIFAR-10, and 0.005–0.013 on SVHN. So the gate chooses class rows only on SVHN, where class means differ by just 10–15% of the within-class spread.

**What it does not do**

The threshold 0.05 was set after seeing SVHN; CIFAR-10 is close to it. On CIFAR-10 both branches were run and give the same accuracy (within 0.13 points), so the choice did not matter there.

## 6 Mechanism 3: the mean-anchored generator

**Purpose**

Remove the only parameters through which a FedAvg-trained generator forgets rare classes, by giving the generator the exact class mean instead of a learned class row.

### 6.1 The failure it removes

In a standard conditional generator, class $c$ enters through a learned row $r_c \in \mathbb{R}^{512}$ (a column of the label projection). It is the only class-specific parameter, and its data gradient is non-zero only on samples of class $c$.

Consider one round. Client $k$ starts from the global row $r_c$, trains locally and returns $r_{k,c}$. Write $\Delta_k = \|r_{k,c}\| - \|r_c\|$, the holders' mean gain $\bar{G}_c = \frac{1}{m_c}\sum_{k\in H_c}\Delta_k$ and the non-holders' mean loss $\bar{D}_c = -\frac{1}{K-m_c}\sum_{k\notin H_c}\Delta_k$, with $p_c = m_c/K$.

**Result 4** (contraction bound)**.** *With plain averaging* $r_c^+ = \frac{1}{K}\sum_k r_{k,c}$*:*

$$\|r_c^+\| \le \|r_c\| + p_c\bar{G}_c - (1 - p_c)\bar{D}_c.$$

*Proof.* Triangle inequality: $\|\frac{1}{K}\sum_k r_{k,c}\| \le \frac{1}{K}\sum_k\|r_{k,c}\| = \|r_c\| + \frac{1}{K}\sum_k\Delta_k$; split the sum into holders ($m_c\bar{G}_c$) and non-holders ($-(K - m_c)\bar{D}_c$). □

**Why non-holders shrink the row fast: Adam with coupled decay.** A client without class $c$ has zero data gradient on $r_c$, so its gradient is pure weight decay, $\lambda r_c$. Adam divides each gradient by the root of its own running average of squares, $\hat{m}/\sqrt{\hat{v}}$, so it is (almost) blind to the size of the gradient: a gradient of $\lambda r_c$ produces a step of about the learning rate $\eta$ toward zero, *whatever* $\lambda$ is. Holders, meanwhile, use $r_c$ to fit their data, which does not push its norm up at full speed. So $\bar{D}_c \gg \bar{G}_c$ and the right-hand side of Result 4 is negative even when most clients hold the class.

Measured on MNIST, where every class is held by at least 7 of 10 clients, the three rarest classes' rows shrank to 4–10% of their initial length.

**A rule from counts alone.** Per coordinate an Adam step is $\eta\psi$ with $\psi \in [-1, 1]$. A non-holder takes $\tau_k$ steps of size $\eta$ toward zero; a holder moves the row only on its $\tau_k f_{kc}$ steps whose batch contains $c$ ($f_{kc}$ = probability that a batch of client $k$ contains class $c$). Hence the ratio

$$\kappa_c = \frac{\sum_{k \in H_c} \tau_k f_{kc}}{\sum_{k \notin H_c} \tau_k}$$

of "steps that can grow the row" to "steps that shrink it". If $\kappa_c < 1$ the averaged drift points to zero for every coordinate: the row collapses. Across 900 logged rows, all 352 rows with $\kappa_c < 1$ collapsed, and a prediction written down before new runs held for 16 of 16 rows.

### 6.2 How the mean anchor works

**Architecture (PC-VAE)**

1. **Anchor.** The exact class means $m_c$ of Section 4 are stored inside the generator as fixed buffers (not trained).
2. **Encoder.** It sees the real feature and its class mean stacked as channels, $[h, m_y]$, and outputs the mean and log-variance of $q(z \mid h, y)$.
3. **Decoder.** It forms $a = W_z z + P m_y$ with a class-*shared* projection $P$, and outputs only the deviation from the class mean:

$$\tilde{h} = \mathrm{ReLU}\left(m_y + \mathrm{dec}(\mathrm{ReLU}(a))\right).$$

4. **Loss.** The usual evidence lower bound: reconstruction error plus $\mathrm{KL}(q(z \mid h, y) \,\|\, \mathcal{N}(0, I))$.
5. **Training.** FedAvg exactly as before (same optimiser, same rounds).
6. **Sampling.** Draw $z \sim \mathcal{N}(0, I)$ and decode with the requested class $y$.

**Why this removes the collapse.** No parameter is specific to a class: $W_z$, $P$ and the decoder are shared by all classes and receive gradients from every client. So non-holders have nothing class-specific to shrink, and Result 4 has nothing to act on. Rare classes also borrow the *shape* of within-class variation from all classes through the shared decoder, while their location comes from their exact mean. (If the decoder were linear, this would be a Gaussian model with exact class means.)

**What it does not do**

The anchor does not force samples to carry their class: when class means are close, the decoder can ignore $m_y$ (Result 3). That is why the gate exists.

## 7 Mechanism 4: class rows with holder-weighted averaging

**Purpose**

On data where class means are nearly identical ($F < 0.05$), keep the generator's learned class rows, which can encode differences the means cannot, but stop them from collapsing.

**Procedure**

The generator is the standard one with class rows $r_c$. All parameters except the rows are averaged by FedAvg. Each row is averaged only over clients that hold the class, weighted by their effective number of samples:

$$r_c \leftarrow \frac{\sum_k E(n_{kc})\, r_{k,c}}{\sum_k E(n_{kc})}, \qquad E(n) = \frac{1-\beta^n}{1-\beta}, \quad \beta = 0.999.$$

**What it does.** Non-holders have $E(0) = 0$, so their decay-only copies are excluded. By the triangle inequality $\|r_c^+\| \le \|r_c\| + \sum_{k\in H_c} \omega_k \Delta_k$ with $\omega_k \propto E(n_{kc})$: the $-(1-p_c)\bar{D}_c$ term of Result 4 disappears. HWA does not force a row to grow; it leaves it to the holders.

**Why weight by $E(n)$.** $E(n)$ grows like $n$ for small $n$ and saturates for large $n$ (at most $1/(1-\beta) = 1000$). It interpolates between weighting holders by their counts ($\beta \to 1$) and weighting them equally ($\beta \to 0$). What matters most is excluding non-holders; the exact weights change little.

**It also fixes a slower problem.** Even without decay, plain averaging dilutes a rare row's learning rate by $m_c/K$ (non-holders contribute no movement but still count in the average), and in the limit of long local training it averages the holders' local means *unweighted*:

**Result 5** (dilution under plain averaging; quadratic row model)**.** *If each holder's local training converges to its local class mean $\bar{t}_{kc}$, plain averaging converges to the unweighted mean of the $\bar{t}_{kc}$, whose mean squared error is*

$$V_c = \frac{n_c}{m_c^2} \sum_{k\in H_c} \frac{1}{n_{kc}} \;\ge\; 1$$

*times that of the pooled mean (equality iff all holders have equal counts), and it gets there $m_c/K$ times more slowly. With weights $E(n_{kc}) \approx n_{kc}$, HWA gives $V_c \approx 1$ and no dilution.*

*Proof sketch.* Non-holders map the row to itself and drop out of the fixed point. The unweighted mean of independent unbiased local means (Result 1) has variance $\frac{1}{m_c^2}\sum_k \sigma^2/n_{kc}$, against $\sigma^2/n_c$ for the pooled mean. □

**Privacy.** The server needs only $\sum_k E(n_{kc}) r_{k,c}$ and $\sum_k E(n_{kc})$, two sums, so HWA runs under secure aggregation and reveals neither any client's row nor its class counts. When all clients have equal counts, HWA is exactly FedAvg.

## 8 Mechanism 5: moment correction

**Purpose**

Generators trained by maximum likelihood tend to produce samples that are too concentrated and, for rare classes, centred in the wrong place. Moment correction moves each class's synthetic samples so that their mean and spread equal the exact class mean $m_c$ and spread $V_c$, while changing them as little as possible.

**Procedure**

Once, after Stage II:

1. For each class $c$, draw 500 samples from the generator and measure their mean $\tilde{m}_c$ and spread $\tilde{V}_c = \mathbb{E}\|\tilde{h} - \tilde{m}_c\|^2$.
2. Compute the scale $s_c = \sqrt{V_c/\tilde{V}_c}$, clipped to $[0.1, 10]$ for safety.

Every time a class-$c$ sample $x$ is drawn afterwards:

$$x' = \mathrm{ReLU}\left(m_c + s_c\,(x - \tilde{m}_c)\right).$$

**The mathematics: the smallest change with the right moments.**

**Result 6** (minimum-displacement correction)**.** *Let $X$ have mean $\tilde{m}$ and spread $\tilde{V}$. Among* all *maps $T$ with $\mathbb{E}\,T(X) = m$ and $\mathbb{E}\|T(X) - m\|^2 = V$, the map $T^\star(x) = m + \sqrt{V/\tilde{V}}\,(x - \tilde{m})$ minimises the average displacement $\mathbb{E}\|T(X) - X\|^2$.*

*Proof.* Let $Y = T(X)$. Expanding around the two means,

$$\mathbb{E}\|Y - X\|^2 = V + \tilde{V} + \|m - \tilde{m}\|^2 - 2\,\mathbb{E}\langle Y - m,\ X - \tilde{m}\rangle.$$

Only the last term depends on how $Y$ is coupled to $X$. By Cauchy–Schwarz, $\mathbb{E}\langle Y - m, X - \tilde{m}\rangle \le \sqrt{\mathbb{E}\|Y - m\|^2\,\mathbb{E}\|X - \tilde{m}\|^2} = \sqrt{V\tilde{V}}$, with equality exactly when $Y - m$ is a positive multiple of $X - \tilde{m}$, i.e. $Y - m = \sqrt{V/\tilde{V}}(X - \tilde{m})$. □

**Reading.** The correction shifts the cloud of samples to the right centre and stretches or shrinks it to the right size, but keeps its *shape*: the directions of variation the generator learned are preserved. It needs no new release: $m_c$ and $V_c$ come from Section 4.

**Worked example 2.** For a rare class the generator's samples have mean $\tilde{m} = (1, 1)$ and spread $\tilde{V} = 0.5$, while the exact statistics are $m = (2, 1)$ and $V = 2$. Then $s = \sqrt{2/0.5} = 2$, and a sample $x = (1.5, 0.5)$ becomes $(2, 1) + 2\,(0.5, -0.5) = (3, 0)$: the cloud moves one unit to the right and doubles its radius.

**What it does not do**

After the final ReLU (features are non-negative) and with moments estimated from 500 samples, the corrected moments match only approximately. Moment correction cannot create class differences that the generator does not produce; it only fixes location and scale.

## 9 Mechanism 6 (optional): kernel herding

**Purpose**

Choose, among many synthetic samples of a class, the subset whose overall distribution is closest to the real class's, as measured by a kernel "fingerprint" that the server can compute exactly.

**The fingerprint.** Random Fourier features turn a feature $h$ into a vector

$$\varphi(h) = \sqrt{2/D}\,\cos(\Omega h + b) \in \mathbb{R}^D, \qquad D = 6144,$$

with the rows of $\Omega$ drawn from Gaussians of three widths $(s_0 \times \{0.5, 1, 2\}$, $s_0 = \sqrt{2\,\overline{V_c}})$ and $b$ uniform on $[0, 2\pi]$. Inner products approximate a Gaussian kernel, $\langle\varphi(h), \varphi(h')\rangle \approx k(h, h')$, and $\|\varphi(h)\| \le \sqrt{2}$. A class's fingerprint is the average $\mu_c = \frac{1}{n_c}\sum_{i:y_i=c}\varphi(h_i)$, released as a sum $\sum_i \varphi(h_i)$ like the other statistics. The distance between two fingerprints is the maximum mean discrepancy (MMD) between the two distributions.

**Procedure**

For each class $c$, when $n$ samples are needed:

1. Draw $4n$ moment-corrected candidates and compute their fingerprints $\varphi_1, \ldots, \varphi_{4n}$.
2. Greedily pick $n$ of them without replacement. With $w_0 = \mu_c$, at step $t$ pick $j_t = \arg\max_i \langle\varphi_i, w_t\rangle$ among unpicked candidates, then update $w_{t+1} = w_t + \mu_c - \varphi_{j_t}$.

*Why this rule.* After $t$ picks, $w_t = (t+1)\mu_c - \sum_{s\le t}\varphi_{j_s}$, which points from the current selection's average toward the target fingerprint. Picking the candidate most aligned with it is the step that most reduces $\|\frac{1}{t}\sum_s \varphi_{j_s} - \mu_c\|$, the MMD between the selection and the real class.

**Motivation.** If a head's loss $\ell$ lies in the kernel's function space with norm at most $B$, then $|\mathbb{E}_{\text{synthetic}}\ell - \mathbb{E}_{\text{real}}\ell| \le B \cdot \text{MMD}$. ReLU networks do not satisfy this premise, so this is a motivation, not a guarantee.

**What to remember**

Kernel herding is consistently positive but small: removing it costs 0.1–0.4 points with the exact readout (positive in 8 of 8 paired seeds), while it makes the head stage about 6× slower. A larger fingerprint or candidate pool changes nothing. It is therefore **optional**: being closer in distribution is not the same as being more useful to the classifier.

## 10 Mechanism 7: logit adjustment in the real phase

**Purpose**

Train each head on its own real data in a way that targets *balanced* accuracy, and, crucially, stop the real phase from erasing what the synthetic phase taught the head about classes this client does not have.

**Procedure**

Client $k$ estimates its class prior with add-one smoothing, $\hat\pi_{kc} = (n_{kc}+1)/(n_k + C)$, which never leaves the client. In the real phase it minimises

$$\ell_{\text{LA}}(h, y) = -\log \frac{\exp\left(f_y(h) + \tau \log \hat\pi_{ky}\right)}{\sum_j \exp\left(f_j(h) + \tau \log \hat\pi_{kj}\right)}, \qquad \tau = 1.$$

At test time the plain logits $f(h)$ are used. The synthetic phase uses unadjusted logits.

**Why it targets balanced accuracy.** Bayes' rule gives $\log p_k(y \mid h) = \log p(h \mid y) + \log \pi_{ky} + \text{const}$, and $p(h \mid y)$ is the same for every client (Result 1). Training $f + \log \hat\pi_k$ to match the client's posterior makes $f$ match $\log p(h \mid y)$, the prior-free score, whose arg-max is the balanced-error-optimal decision.

**What it cannot do on its own.**

**Result 7** (logit corrections are reparametrisations)**.** *For a head with a free last-layer bias $b$ trained by SGD, training with $\ell(f + a_k, y)$ from $(\theta_0, b_0)$, for any fixed vector $a_k$, gives exactly the same trajectory as training with $\ell(f, y)$ from $(\theta_0, b_0 + a_k)$: every parameter except $b$ is identical at every step, and $b_t^{\text{LA}} = b_t^{\text{CE}} - a_k$.*

*Proof.* $f + a_k = We + (b + a_k)$, and $b \mapsto b + a_k$ is a translation with identity Jacobian, so the gradients at corresponding points agree; induct over steps. □

So for a head trained on real data alone, logit adjustment changes only the bias: it is a readout correction, and the exact readout (Section 12) makes it redundant there. Its real job is during training, because of how the two phases alternate:

**Result 8** (what the alternation keeps; quadratic losses)**.** *Let the synthetic and real losses be quadratic with Hessians $H_s, H_r$ and minimisers $\theta_s, \theta_r$, and let each round take $S_s$ synthetic then $S_r$ real gradient steps, $P_s = (I - \eta H_s)^{S_s}$, $P_r = (I - \eta H_r)^{S_r}$. The rounds converge to*

$$\theta^\star = \theta_r + \Omega(\theta_s - \theta_r), \qquad \Omega = (I - P_r P_s)^{-1} P_r (I - P_s).$$

*Along an eigendirection $i$ (commuting Hessians) the weight of the synthetic solution is* $\omega_i = \frac{\rho_{r,i}(1-\rho_{s,i})}{1-\rho_{r,i}\rho_{s,i}} \le \rho_{r,i} = (1-\eta h_{r,i})^{S_r}$.

*Proof.* A round is the affine map $\theta \mapsto P_r P_s \theta + P_r(I-P_s)\theta_s + (I-P_r)\theta_r$; solve for its fixed point. □

**Reading.** Where the real loss is flat ($h_{r,i} = 0$, so $\rho_{r,i} = 1$), the synthetic phase decides ($\omega_i = 1$); where the real loss is curved, the real phase overwrites it. The flat directions are exactly those of classes the client lacks, which are the ones balanced accuracy needs. But with plain cross-entropy the real loss is *not* flat there: it keeps pushing the logits of absent classes down without limit (as on separable data), so the real phase erases what the synthetic phase put in. With logit adjustment, absent classes have tiny adjusted probability ($\log \hat{\pi}_{kc}$ very negative), their gradient is $O(1/(n_k + C))$, and the synthetic knowledge survives. Measured: the share of the synthetic phase's change to absent-class rows that survives the next real phase is 0.96–1.0 with logit adjustment and 0.76–0.98 without.

# 11 Mechanism 8: consensus soft labels on synthetic features

**Purpose**

Give each synthetic feature a target that reflects what it actually looks like, not only the class it was generated for, so that imperfect synthetic features teach less wrong information.

## 11.1 Why synthetic labels matter: the signal fraction

**Result 9** (label-free synthetic data penalise the spread of the logits)**.** *If a synthetic feature's label $y$ is uniform on the $C$ classes and independent of the feature, then for every logit vector $f$, with $\bar{f} = \frac{1}{C}\sum_c f_c$,*

$$\mathbb{E}_y\, \ell(f, y) = \mathrm{LSE}(f) - \bar{f} = \log C + \mathrm{KL}\left(u \,\|\, \mathrm{softmax}\, f\right) = \log C + \frac{1}{2C}\sum_c (f_c - \bar{f})^2 + O(\|f - \bar{f}\|^3).$$

*Proof.* Cross-entropy is $\mathrm{LSE}(f) - f_y$; averaging over a uniform $y$ gives $\mathrm{LSE}(f) - \bar{f}$. The KL divergence from the uniform distribution $u$ to softmax $f$ is $-\log C + \mathrm{LSE}(f) - \bar{f}$. Expand LSE around equal logits for the quadratic term. □

**Reading.** Synthetic features that carry no class information do not teach nothing: they teach the head to make all logits equal on them, i.e. to be unsure, exactly in the region where the real data are (the generator was fitted to them). If a fraction $s$ of synthetic features carries its label (the *signal fraction*) and the rest do not, the synthetic loss is $s \cdot$(useful cross-entropy)$+(1-s)\cdot$(penalty above). So the synthetic phase helps when $s$ is high and harms when $s \approx 0$; measured, the gain changes sign between $s \approx 0.03$ and 0.07. The anchor and moment correction raise $s$; consensus labels lower the damage of the part that remains wrong.

### 11.2 How consensus labels work

**Procedure**

Every Stage-III round:

1. The server draws one shared pool of synthetic features $\tilde{h}$ with uniformly random classes $y$ (from a public seed, so clients can regenerate it).
2. Each architecture's current head $h_g$ evaluates the pool; the server averages their probabilities (a sum over architectures, so it fits secure aggregation):

$$\bar{p}(\tilde{h}) = \frac{1}{G}\sum_{g=1}^{G} \operatorname{softmax} h_g(\tilde{h}).$$

3. The target for each synthetic feature is

$$t(\tilde{h}) = (1-\beta)\, e_y + \beta\, \bar{p}(\tilde{h}), \qquad \beta = 0.5,$$

where $e_y$ is the one-hot vector of its generated class.

4. Each client trains its head for $T_s$ epochs on its slice of the pool with the soft cross-entropy $-\sum_c t_c(\tilde{h}) \log \operatorname{softmax}_c f(\tilde{h})$.

**Why mix, and why $\beta = \frac{1}{2}$.** The best possible target for a feature is its true posterior $p(\cdot \mid \tilde{h})$; the one-hot label is off by the generator's error. The ensemble of $G$ heads of different architectures, trained on different clients, approximates that posterior with errors that partly cancel (variance down by up to $1/G$). Keeping half of the one-hot label bounds the damage if the ensemble shares an error. A stylised model makes the trade-off explicit:

**Result 10** (label error limits the useful amount of synthetic data)**.** *A head estimates a parameter from $n_r$ real samples (variance $\sigma^2$ each) and a fraction $\lambda$ of $n_s$ synthetic samples whose labels have bias $b$. Writing $w = \frac{\lambda n_s}{n_r + \lambda n_s}$, the mean squared error is $\mathcal{E}(w) = \frac{\sigma^2(1-w)}{n_r} + w^2 b^2$, minimised at $w^\star = \frac{\sigma^2}{2n_r b^2}$ (clipped to the feasible range). The optimal synthetic share falls as the label bias $b$ grows, and reaches all available synthetic data as $b \to 0$.*

*Proof.* $\mathcal{E}$ is a convex parabola in $w$; set its derivative to zero. □

**Reading.** With hard labels, more synthetic epochs eventually hurt, because the label bias does not average away. Moving the target toward the posterior lowers $b$, so the same synthetic budget becomes useful. This is why consensus labels and a larger $T_s$ work together.

**Cost.** The server broadcasts the pool seed and $P \times C$ soft labels per round (about 24 KB here). No real data are used.

## 12 Mechanism 9: the exact readout

**Purpose**

Replace each architecture's last layer, which is biased toward its own clients' class mix, by the class-balanced linear classifier computed from exact sums over *all* clients. This removes the single largest loss.

**Where the formula comes from.** Suppose that, in the embedding space $e = \text{body}(h)$ of one architecture, each class is Gaussian with its own mean $\mu_c$ and a shared covariance $\Sigma$, and that we want to treat all classes as equally likely (balanced accuracy). Then

$$\log p(e \mid c) = -\tfrac{1}{2}(e-\mu_c)^\top \Sigma^{-1}(e-\mu_c) + \text{const} = \mu_c^\top \Sigma^{-1} e - \tfrac{1}{2}\mu_c^\top \Sigma^{-1}\mu_c + (\text{terms without } c),$$

which is linear in $e$: a last layer with $w_c = \Sigma^{-1}\mu_c$ and $b_c = -\frac{1}{2}\mu_c^\top \Sigma^{-1}\mu_c$. This is linear discriminant analysis with a uniform class prior.

**Procedure**

Once, after Stage III, for each architecture $g$:

1. The server broadcasts the body of architecture $g$ to every client (bodies are small).
2. Each client embeds its real features, $e = \text{body}_g(h)$, and uploads per-class counts $n_c$, per-class sums $\sum e$ and the total second moment $M = \sum ee^\top$.
3. The server computes class means $\mu_c$ and the pooled within-class covariance

$$S = \frac{M - \sum_c n_c \mu_c \mu_c^\top}{N - C},$$

shrinks it toward a multiple of the identity for stability,

$$S_\gamma = (1-\gamma)S + \gamma \frac{\operatorname{tr} S}{d} I, \qquad \gamma = 0.1,$$

and sets the last layer to $w_c = S_\gamma^{-1}\mu_c$, $b_c = -\frac{1}{2}\mu_c^\top S_\gamma^{-1}\mu_c$.

*Derivation of $S$.* $\sum_c \sum_{i \in c}(e_i - \mu_c)(e_i - \mu_c)^\top = \sum_i e_i e_i^\top - \sum_c n_c \mu_c \mu_c^\top$, divided by the degrees of freedom $N - C$.

**Why it is exact.** Everything the server uses is a sum, so by Result 2 the readout equals the one a centralised learner would compute for the same bodies. The federated last layer then stops being a source of loss relative to centralised training. It never uses the trained last layer, so its accuracy is a direct measure of how well the *body* separates the classes. It also makes logit adjustment's bias correction redundant (Result 7), but not its training role (Result 8).

**Cost.** One extra round; per client, up to about $5 \times 10^5$ numbers per architecture (the symmetric $d \times d$ second moment for $d \leq 1024$).

**What it does not do**

It is only as good as the body: it removes the last-layer loss, not the representation's. The linear-Gaussian model is an approximation; shrinkage keeps it stable for rare classes.

## 13 Optional: noise on the released sums

**Purpose**

Bound what the released class statistics reveal about any individual sample, at a known cost in accuracy.

Each feature is clipped to norm at most $c_{\max}$, $h \mapsto h \min(1, c_{\max}/\|h\|)$, so one sample changes $n_c$ by 1, $S_c$ by at most $c_{\max}$, $Q_c$ by at most $c_{\max}^2$ and the fingerprint sum by at most $\sqrt{2}$. Gaussian noise $\sigma\times$(that sensitivity) is added to each release, with $\sigma = \sqrt{2/\rho}$ and the privacy budget $\rho = \left(\sqrt{\log(1/\delta) + \varepsilon} - \sqrt{\log(1/\delta)}\right)^2$ (zero-concentrated differential privacy, split equally over the releases). A class whose noisy statistic is too noisy relative to its size simply skips moment correction and herding. The guarantee covers the added statistics only, not the FedAvg training of the generator or heads.

# 14 Putting it together

**The full algorithm**

1. **Stage I.** Train the feature extractor $\phi$ by FedAvg; freeze it.
2. **Release** (Section 4). Secure-aggregate $n_c$, $\sum h$, $\sum \|h\|^2$ per class (and $\sum \varphi(h)$ if herding is used). Server: $m_c$, $V_c$, $F$ (and $\mu_c$).
3. **Gate** (Section 5). If $F \geq 0.05$, build the mean-anchored generator with the $m_c$ as buffers; else the class-row generator.
4. **Stage II** (Sections 6 and 7). Train the generator by FedAvg; with class rows, average each row by HWA.
5. **Sampler** (Sections 8 and 9). Measure $\tilde{m}_c$, $\tilde{V}_c$ from 500 samples per class; from now on every sample is moment-corrected (and, optionally, herded).
6. **Stage III**, for each round:
   (a) Server: draw the shared synthetic pool; collect every architecture's probabilities on it; form consensus targets $t = \frac{1}{2}e_y + \frac{1}{2}\bar{p}$ (Section 11).
   (b) Each client: $T_s$ epochs on its slice of the pool with soft cross-entropy; then $T_r = 5$ epochs on its real features with logit adjustment by its own prior (Section 10).
   (c) Server: average heads within each architecture.
7. **Exact readout** (Section 12). For each architecture, secure-aggregate $n_c$, $\sum e$, $\sum ee^\top$ of its embeddings over all clients; replace the last layer by $S_\gamma^{-1}\mu_c$, $-\frac{1}{2}\mu_c^\top S_\gamma^{-1}\mu_c$.

**What each problem meets.**

| Problem | Mechanisms that address it |
|---|---|
| Rare classes collapse in the generator | mean anchor (no class rows); HWA when rows are needed |
| Rare-class samples in the wrong place or too narrow | moment correction (exact mean and spread) |
| Classes the generator cannot separate | gate on $F$, falling back to class rows |
| Synthetic features with wrong labels | consensus soft labels; higher signal fraction from anchor + correction |
| Real phase erases knowledge of absent classes | logit adjustment by the client's own prior |
| Last layer fitted to one client's class mix | exact readout from sums over all clients |

**What travels, and when.**

| Message | When | Content |
|---|---|---|
| Class statistics | once, after Stage I | $C(d+2)$ numbers per client (plus $CD$ for herding), summed |
| HWA sums | each Stage-II round (rows only) | $\sum_k E(n_{kc})r_{k,c}$ and $\sum_k E(n_{kc})$ |
| Consensus probabilities | each Stage-III round | $P \times C$ per architecture, summed |
| Exact-readout sums | once, at the end | $n_c$, $\sum e$, $\sum ee^\top$ per architecture |

No raw image, real feature, label or single-client statistic ever leaves a client.

# 15 Cheat sheet

| Symbol | Meaning |
|---|---|
| $h = \phi(x)$ | feature of an image from the frozen extractor |
| $n_{kc},\ n_c,\ N$ | class-$c$ count on client $k$, over all clients, total |
| $\hat{\pi}_{kc}$ | client $k$'s smoothed class prior, $(n_{kc}+1)/(n_k+C)$ |
| $m_c,\ V_c$ | exact class mean and spread of features |
| $F$ | class separation: mean squared distance of class means from their centre over mean spread |
| $r_c$ | class row of the class-row generator |
| $H_c,\ m_c$ (count) | clients holding class $c$, and their number |
| $\kappa_c$ | steps that can grow row $c$ over steps that shrink it; $< 1$ means collapse |
| $E(n)$ | effective number of samples, $(1-\beta^n)/(1-\beta)$ |
| $\tilde{m}_c,\ \tilde{V}_c,\ s_c$ | generator's class mean and spread; correction scale $\sqrt{V_c/\tilde{V}_c}$ |
| $\varphi,\ \mu_c$ | random-feature map and exact class fingerprint (herding only) |
| $s$ | signal fraction: share of synthetic features that carry their label |
| $\bar{p}(\tilde{h}),\ t(\tilde{h})$ | averaged head probabilities; consensus target |
| $e,\ \mu_c,\ S_\gamma$ | embedding of a head's body; its class mean; shrunk within-class covariance |

| Setting | Value |
|---|---|
| Gate threshold on $F$ | 0.05 |
| Generator latent size | 16 (50 on CIFAR) |
| Generator training | 5 local epochs per round, Adam, learning rate $10^{-3}$, weight decay $10^{-3}$ |
| HWA | $\beta = 0.999$ |
| Moment correction | 500 samples per class; scale clipped to $[0.1, 10]$ |
| Kernel herding (optional) | $4\times$ candidates; $D = 6144$ random features, three widths |
| Logit adjustment | $\tau = 1$, add-one smoothed client prior |
| Consensus labels | $\beta = 0.5$; one shared pool per round |
| Stage-III epochs | $T_s$ synthetic (chosen on validation data; 10 or 1), $T_r = 5$ real; SGD, learning rate 0.1 |
| Exact readout | shrinkage $\gamma = 0.1$ |